Follow-up question (applied skill): "What problem does the React Context API solve, and how does it compare to traditional prop drilling?"

Analysis: This question evaluates a candidate's ability to recognize design patterns for state sharing in React, understand trade-offs between different approaches, and articulate how React's Context API simplifies component communication. It tests not only technical knowledge but also the candidate's skill in balancing maintainability, performance, and scalability in UI architecture.

Answer: The React Context API addresses a common problem in component-based architectures known as prop drilling. Prop drilling occurs when data needs to be passed through multiple intermediate components that do not directly use it, simply to reach a deeply nested child component. This approach can lead to bloated, hard-to-maintain code, as every intermediate component is forced to accept and forward props unnecessarily.

In listing 4.2, a `data` string is declared in `ParentComponent` and passed through `ChildA` and `ChildB` before finally being used in `ChildC`. Each component along the way must explicitly accept and forward the data prop—even though only `ChildC` actually consumes it.

Listing 4.2 Prop drilling example

```
const ParentComponent: React.FC = () => {
  const data = 'Hello from Parent!';
  return (
    <div className='prop-drilling'>
      <h1>Parent Component</h1>
      <ChildA data={data} /> #A
    </div>
  );
};
const ChildA: React.FC<DataProps> = ({ data }) => {
  return (
    <div>
      <h2>Child A Component</h2>
      <ChildB data={data} /> #B
    </div>
  );
};
```